to BSTs. In binary search trees we replace the removed node with its successor (or predecessor), but this approach wouldn't work in treaps, because this replacement could have a smaller priority than its new children, and in that case, it would need to be pushed down. Moreover, in the general case for BSTs the successor of a node is not a leaf, and so it needs to be recursively removed.

Listing 3.6 method Treap::remove

```
function remove(treap, key)
                                    #1
  node ← search(treap.root,
key)                                                  #2
  if node == null
then                                                            #3
    return false
  if isRoot(node) and isLeaf(node)
then                                             #4
    treap.root ← null
    return true
  while not isLeaf(node) do
                                                     #5
    if node.left != null
        and (node.right==null
          or node.left.priority < node.right.priority) then
         #6

rotateRight(node.left)
    #7
    else

 rotateLeft(node.right)
     #8
    if isRoot(node.parent)
then                                                  #9
      treap.root ←
node.parent
  if node.parent.left == node
then                                               #10
```